

Camera: shopper interaction detected
Load cell: -150 g
→ likely physical product interaction

Add ToF

Camera: shopper near shelf
ToF: 80 mm
Load cell: -150 g
→ stronger evidence of a shelf interaction

Add IR break-beam

IR beam: crossing event
Camera: person detected
→ stronger entrance/exit timing

Add environmental sensing

BME280/BH1750
→ store context, not inventory truth
→ useful for diagnostics/contextual analytics

Design rule: A sensor belongs on the PCB only if you can explain what ambiguity it removes or what useful decision it improves.

4. Recommended Blue Pill Pin/Interface Strategy

The exact final pinout should be frozen only after the selected Blue Pill board and sensor modules are physically tested. A modular connector strategy is preferable for the first PCB revision.

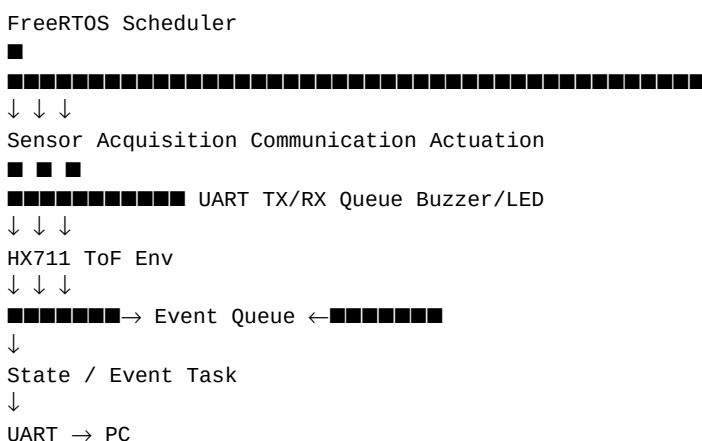
STM32F103C8T6

HX711 → dedicated GPIO pair
ToF → I²C
BME280 → I²C (shared bus if address permits)
BH1750 → I²C (shared bus if address permits)
PIR → GPIO input
IR beam → GPIO input / interrupt-capable pin
Buzzer → GPIO / transistor driver
LED → GPIO
UART → edge computer
SWD → programming/debug

For a PCB, use labelled connectors and test points. Keep optional sensors on headers so the same board can run a minimal configuration or an expanded configuration.

5. FreeRTOS Architecture

FreeRTOS changes the firmware from a sequential polling program into a small real-time system with independent tasks communicating through queues, semaphores and notifications.



6. Suggested FreeRTOS Tasks

Task	Responsibility	Typical behavior
HX711 Task	Sample load cell	Periodic sampling; filter/average; publish weight event
ToF Task	Read distance sensors	Periodic reads; detect near/far transitions
Presence Task	PIR / IR	Wait for GPIO event or poll at low rate
Environment Task	BME280/BH1750	Slow periodic sampling
Fusion/State Task	Build sensor-node state	Consume events; timestamp; detect local patterns
UART Task	Communicate with edge PC	Transmit structured sensor messages
Actuator Task	Buzzer/LED/servo	Consume commands; execute non-blocking patterns
Health Task	Watchdog/heartbeat	Monitor task health and communication

7. Example FreeRTOS Priority Model

Priorities should be chosen after measuring timing requirements. The following is an architectural example, not a mandatory configuration.

```
Higher priority
■
■■ Actuation / safety response
■■ UART receive / command handling
■■ HX711 sampling
■■ ToF / fast presence events
■■ Sensor-state processing
■■ Environment sensing
■■ Diagnostics / logging
■
Lower priority
```

Do not assign high priority to every task. The benefit of an RTOS comes from predictable scheduling and sensible prioritization.

8. Queues and Event-Driven Design

Instead of every task directly manipulating shared state, sensor tasks should publish timestamped events into a queue. A central state task can consume them.

```
Sensor event example:
TYPE=HX711
TIMESTAMP=...
VALUE=321290
```

```
TYPE=TOF
ZONE=A
DISTANCE=82
```

```
TYPE=IR
EVENT=ENTRY
```

```
TYPE=BME280
TEMP=...
HUMIDITY=...
```

This creates a clean boundary between firmware sensing and the PC-side intelligence.

9. Why FreeRTOS Is Valuable for StoreMind

- Multiple sensors can operate at different sampling rates.
- UART communication can run independently of sensor acquisition.
- Actuation does not need to block sensor acquisition.
- Sensor failures can be isolated more cleanly.
- Tasks can have independent timing requirements.
- The architecture is easier to extend when adding sensors.

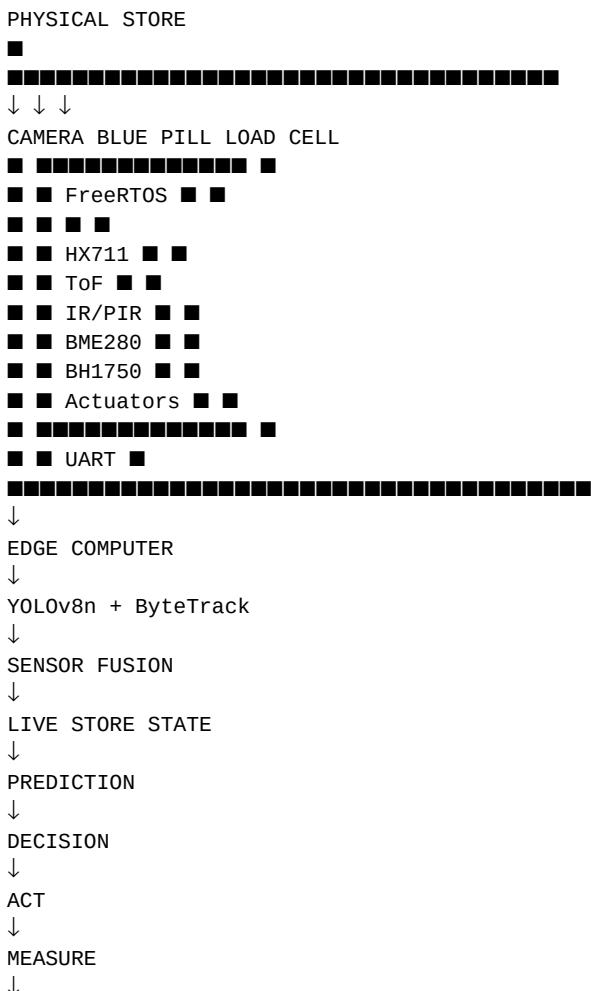
Panel answer: “We use FreeRTOS because the sensor node is performing several concurrent real-time responsibilities—sampling, communication and actuation. Tasks can run at their own rates and communicate through queues instead of one blocking super-loop.”

10. Example Timing

Every ~10-20 ms → fast sensor/event processing
Every ~50-100 ms → ToF / interaction sensing
Every ~100 ms → HX711 processing/filter update
Every ~1 s → environmental sensors
Continuous → UART communication
Event-driven → buzzer/LED response
Periodic → health/heartbeat

These intervals are illustrative. Final rates should be selected from the actual sensors, required response time and measured CPU utilization.

11. End-to-End Multimodal Architecture



LEARN

12. Example: One Retail Event Across the Whole Stack

T0 Camera: shopper enters Shelf A ROI
T1 ToF: distance falls to 80 mm
T2 HX711: load decreases 150 g
T3 FreeRTOS tasks timestamp and transmit events
T4 Backend receives events
T5 Fusion: HIGH confidence interaction
T6 Shopper returns product
T7 HX711 returns near baseline
T8 Fusion: interaction confirmed, no permanent depletion
T9 StoreMind records product-interest event

13. Example: Missed Camera Event

Camera: no pickup detected
HX711: -150 g
ToF: 82 mm
↓
Multimodal disagreement
↓
Possible missed visual event
↓
Lower confidence / investigate
↓
Physical evidence preserved

This is a particularly strong demonstration of why multimodal sensing exists.

14. Example: Stockout Prediction

High footfall
+ repeated shelf interactions
+ declining load
+ historical consumption
↓
Stockout risk HIGH
↓
Replenishment action
↓
Load restored
↓
Risk falls
↓
Intervention successful

15. FreeRTOS + PCB + Valkey Relationship

BLUE PILL / FreeRTOS
↓ structured sensor events
UART
↓
EDGE COMPUTER / FASTAPI
↓
VALKEY (optional)
■ current sensor state
■ recent events
■ cached dashboard state
↓
SSE / API
↓
REACT DASHBOARD

Valkey belongs on the edge-computer/backend side, not on the Blue Pill. FreeRTOS belongs on the STM32 firmware side.

16. Resource and Scope Reality

The STM32F103C8T6 is a small microcontroller. FreeRTOS is feasible for a sensor node, but the firmware should remain lean. Do not attempt to run the computer-vision pipeline, large AI models or the React/backend stack on the Blue Pill.

- Keep task stacks small and measured.
- Avoid unnecessary dynamic allocation.
- Use queues for structured events.
- Use timeouts so one failed sensor cannot block the whole system.
- Use a watchdog/heartbeat for robustness.
- Log only what is needed over UART.

17. Recommended PCB Revision Strategy

PCB V1 — Hackathon-safe

- STM32F103C8T6 module/header
- HX711 + load-cell connector
- UART connector
- SWD header
- Power regulation and protection
- Status LED + reset + BOOT0
- Buzzer connector
- Optional I²C header

PCB V2 — Expanded sensor node

- Multiple ToF connectors
- IR break-beam inputs
- PIR input
- BME280/BH1750 I²C devices
- Additional actuator outputs
- More robust power/ESD protection
- Dedicated debug/test points

Recommendation: Make V1 modular. Do not cram every possible sensor onto the first board simply because FreeRTOS makes them possible.

18. How This Strengthens the Project

- Embedded systems depth: the Blue Pill becomes a real sensor node rather than a serial bridge.
- Real-time systems depth: FreeRTOS introduces scheduling, tasks, queues, synchronization and fault handling.
- Sensor-fusion depth: multiple independent measurements are time-aligned and combined.
- Hardware depth: the PCB integrates sensing and actuation.
- Systems depth: the STM32, edge AI, backend and UI have clean responsibilities.

- Scalability: the same sensor-node architecture can support multiple shelves.

19. What You Should Actually Implement in 10 Days

Do not implement every sensor + FreeRTOS task just to say you did. The strongest version is a stable core with one polished real-time sensor node.

- Real HX711 task on STM32/Blue Pill
- UART event protocol
- One FreeRTOS communication task
- One actuator task
- Optional ToF interface if hardware is available
- Health/heartbeat task
- PC-side sensor fusion
- Prediction + decision + outcome loop
- React visualization
- PCB schematic/layout as a documented hardware refinement

Additional BME280/BH1750/PIR/IR sensors can be presented as an extensible architecture or added only if they directly support a demo scenario.

20. Panel Questions for the FreeRTOS Version

Why FreeRTOS instead of a simple while loop?

“A simple loop works for one or two sensors, but our sensor node has concurrent acquisition, communication and actuation requirements. FreeRTOS gives each responsibility predictable scheduling and lets us communicate through queues without blocking unrelated tasks.”

Why is the Blue Pill not running the AI?

“The Blue Pill is the deterministic sensing and actuation node. Vision inference is computationally heavier, so it runs on the edge computer. This separation keeps the embedded node low-cost and reliable.”

Why add sensors if you already have a camera?

“We only add a sensor when it provides independent evidence or resolves an ambiguity. More sensors are useful because different physical phenomena are being measured, not because more sensors automatically mean better AI.”

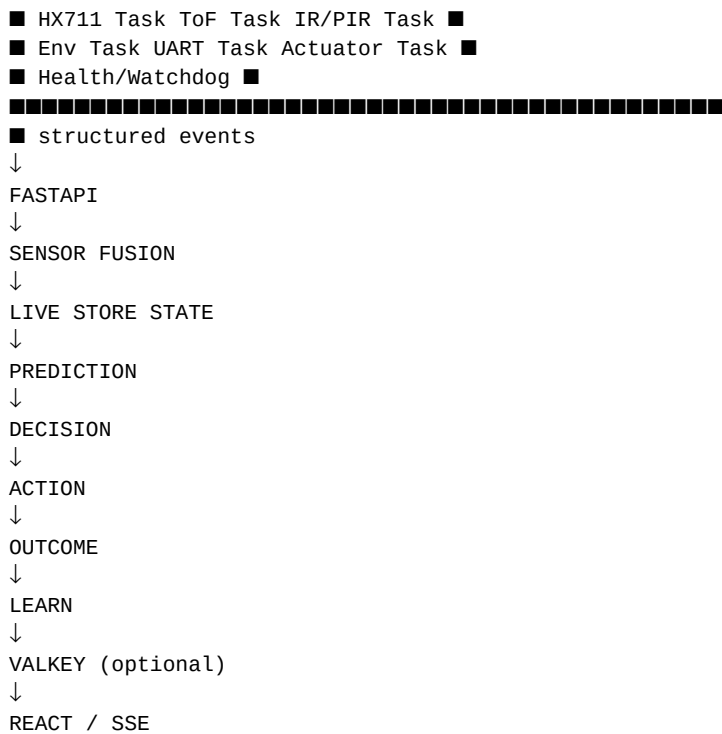
What is the role of FreeRTOS in sensor fusion?

“FreeRTOS handles acquisition and timing at the sensor node. It timestamps and packages observations; the higher-level fusion engine on the edge computer combines them with computer-vision events.”

21. Final Expanded Architecture

STOREMIND
SELF-OBSERVING RETAIL STORE

[illegible]



22. Final Positioning

Core StoreMind: camera + HX711 + edge AI + fusion + prediction + decision + feedback.

Advanced StoreMind: custom STM32 sensor node + FreeRTOS + modular sensors + structured event bus + PCB + optional Valkey infrastructure.

Important: the advanced architecture should demonstrate engineering depth without replacing the core intelligence loop.

This addendum should be read together with the StoreMind Complete Project Guide. It deliberately distinguishes the current working prototype from potential sensor and RTOS extensions so that the project remains technically honest and defensible.